

Agentic Development using AWS

for Airline Data Pipelines & Real-Time Inference

Course Outline, Pre-Reads, Case Studies, Lab Setup

Item	Detail
Prepared for	Accelya Kale
Delivered by	Jai Singh
Program duration	48 hours 12 half-day modules × 4 hours each
Cadence	Half-day delivery (client preference) so teams can work the other half
Tentative start	11 May 2026
Mode	Instructor-led, hands-on labs
Tooling stack	AWS + Microsoft (VS Code / Windsurf, GitHub Copilot, Azure OpenAI)
Lab strategy	Trainer-managed AWS sandbox on synthetic airline data
Document version	v1.1 17 April 2026

Contents

1. Executive Summary & Design Intent
2. Audience, Tooling & Ground Rules
3. Program Structure at a Glance
4. Detailed Table of Contents (12 Half-Day Modules)
5. Pre-Reads & Participant Readiness
6. Case Studies & Business Scenarios
7. Lab Setup Guide
8. Synthetic Data | Ownership, Scope & Open Question
9. Deliverables, Assumptions & Next Steps

1. Executive Summary & Design Intent

This proposal outlines a revised 48-hour instructor-led program, **Agentic Development using AWS for Airline Data Pipelines & Real-Time Inference**, designed for Accelya Kale and delivered by Jai Singh. Per your preference, the program is delivered as **12 half-day modules of 4 hours each**. The original scoping note focused on a single S3 + Lambda agentic flow. Based on the email from your training partner and the airline glossary and demo flows you shared, the scope has been expanded to cover the full lifecycle your teams actually work on — batch pipelines, streaming pipelines, and real-time ML/DL inference — while keeping the agentic, AI-assisted engineering approach as the red thread through every module.

Design intent in one paragraph

Participants will finish this program able to design, build, and review an AWS-based airline data platform using an agentic workflow — where AI generates architecture options, scaffolding, and tests; human engineers act as expert reviewers; and every artefact is modular, observable, cost-aware, and production-minded. The program is hands-on first, with roughly 55–60% of each 4-hour module spent in labs, 25% in guided design and review, and 15–20% in concept explanation.

2. Audience, Tooling & Ground Rules

2.1 Audience profile

Based on the context shared, the batch has three experience bands. The program treats this explicitly rather than averaging everyone into one track.

Band	Profile	Role in class
Foundation (≤ 1 yr)	Limited AWS exposure, may be comfortable with general programming or basic DevOps.	Pairs with an experienced learner during labs; Modules 1–3 bring them to baseline.
Working (3+ yrs)	Comfortable with AWS fundamentals, has built pipelines, new to agentic AI-assisted workflows.	Drives lab implementation; contributes to design reviews.
DevOps / Senior (3.5+ yrs)	Strong operational and pipeline background, likely familiar with IAM, CI/CD, observability.	Leads architecture critique; reviews IAM, cost, and production-readiness of peer work.

2.2 Tooling stack (Microsoft + AWS only)

All development, review, and AI-assisted activities use the approved stack. No Google services and no non-approved OpenAI endpoints are used in any lab.

Layer	Tool	Purpose in this program
IDE	VS Code (primary) or Windsurf	Editor of record; both run the GitHub Copilot extension. Trainer demonstrates in VS Code; learners may use Windsurf if preferred.
AI code assistance	GitHub Copilot / Copilot Chat	Inline completion, chat-based refactoring, and test generation inside the IDE.
Agentic reasoning model	Azure OpenAI (via Azure AI Foundry) Or AWS Bedrock	Architecture options, BRD drafting, prompt-driven scaffolding.
Source control	GitHub / GitHub Codespaces (optional)	Starter repository, branch-per-learner, PR-based reviews.
Cloud platform	AWS	S3, Lambda, Kinesis, Glue, Athena, CloudWatch, SageMaker Serverless Inference, IAM.

Layer	Tool	Purpose in this program
Infrastructure as code	AWS SAM or CloudFormation	Deterministic deploys for labs; no click-ops in production-shaped exercises.

2.3 Ground rules for the workshop

- Hands-on first. Each 4-hour module is roughly 2 hours 15 minutes of labs, 1 hour of guided design, and 45 minutes of concept + review.
- Human-in-the-Loop is mandatory. No AI-generated code is merged without a human review pass using the course review checklist.
- Airline vocabulary is used in every prompt and artefact. Generic “order” or “payment” examples are replaced with PNR, fare class, and settlement.
- Synthetic data only. No real Accelya customer or production data is used at any point — see Section 9.
- End-of-module cleanup. Participants tear down ephemeral resources at the end of each module to keep the sandbox predictable and costs bounded.

3. Program Structure at a Glance

The program is delivered as **12 half-day modules of 4 hours each**, totaling 48 hours. Modules 1–3 build the shared baseline; Modules 4–10 each cover one technical slice (batch, analytics, streaming, inference); Modules 11–12 deliver the integrated capstone and evaluation.

#	Module	What participants build	Track
1	Program Foundation & the Agentic Engineering Model	Shared operating model for the course; first AI-assisted architecture option set from an airline prompt.	Foundation
2	AWS Fundamentals for the Workshop	Working AWS sandbox; verified IAM access; inspected S3 layout; first Lambda deployed and read from CloudWatch.	Foundation
3	Agentic AI Foundations for Engineering Teams	Prompt and context patterns for engineering; two architecture variants generated with HITL review.	Foundation
4	Batch Pipeline Design on AWS	Raw → validated → curated S3 zones; design for an airline ticket-issue batch pipeline.	Batch
5	Agentic Batch Processing Implementation	AI-assisted BRD + scaffolded Lambda processor with validation, structured logging, and unit tests.	Batch
6	Data Transformation, Cataloguing & Analytics	Curated Parquet dataset in Glue Data Catalog; Athena reconciliation & exception queries.	Batch → Analytics
7	Streaming Architecture on AWS	Kinesis stream ingesting synthetic booking, cancellation, refund events with a Lambda consumer.	Streaming
8	Agentic Real-Time Processing Patterns	AI-compared consumer patterns; modular streaming processor with retries and DLQ.	Streaming
9	Real-Time ML/DL Inference on AWS	SageMaker Serverless endpoint invoked from a streaming Lambda for refund recommendation / anomaly flagging.	Inference

#	Module	What participants build	Track
10	Observability, Security & Cost Optimisation	Alarms, dashboards, tightened IAM, and a cost guardrail checklist applied to the built solution.	Production-readiness
11	End-to-End Airline Case Study	Booking → ticketing → cancellation → refund → settlement integrated on the learner sandbox.	Capstone
12	Capstone Review & Expert Evaluation	Peer review, production-readiness scorecard, improvement backlog, individual readiness check.	Capstone

3.1 Time split within each 4-hour module

Block	Duration	Typical use
Concept briefing	30–45 min	Topic explanation, reference architecture walkthrough, tie-in to airline flows.
Guided design	45–60 min	AI-driven architecture option generation, HITL review, selection.
Hands-on lab	2 hrs – 2 hrs 15 min	Implementation, testing, and debugging in VS Code / Windsurf on the AWS sandbox.
Review & wrap	20–30 min	Peer review, cost / IAM / observability checklist, end-of-module cleanup.

3.2 Suggested delivery cadence

A 4-hour module can be delivered in a morning (e.g. 09:30–13:30) or afternoon (e.g. 14:00–18:00) slot, based on Accelya's preference.

4. Detailed Table of Contents (12 Half-Day Modules)

Each module below lists the concepts covered, the hands-on labs, the airline vocabulary used during that module, and the expected outputs.

Module 1 — Program Foundation & the Agentic Engineering Model

Field	Detail
Duration	4 hours ~30–45 min concepts, ~45–60 min guided design, ~2–2.25 hrs labs, ~20–30 min review
Track	Foundation
Module goal	Establish the operating model for the program — what agentic development means, how HITL review fits engineering work, and how the course uses airline vocabulary throughout.

Concepts covered

- What agentic development means inside an enterprise engineering team and how it differs from “just using Copilot”.
- Human-in-the-Loop (HITL) architecture: prompt → options → review → build → validate.
- Workshop operating model: how each module will flow, what a lab deliverable looks like, what a good review looks like.
- Batch vs streaming vs real-time inference — when each is appropriate in an airline data platform.
- How airline vocabulary will be embedded in prompts, payloads, schemas, and labs across every module.

Hands-on labs

- Open the course starter repository in VS Code / Windsurf; verify GitHub Copilot and Azure OpenAI access.
- Convert a one-line airline business requirement into an initial technical prompt.
- Ask Azure OpenAI for two high-level architecture options; review as a group using the course HITL checklist.

Module 2 — AWS Fundamentals for the Workshop

Field	Detail
Duration	4 hours ~30–45 min concepts, ~45–60 min guided design, ~2–2.25 hrs labs, ~20–30 min review

Field	Detail
Track	Foundation
Module goal	Bring the mixed-experience batch to a common AWS working baseline across the services this program actually uses.

Concepts covered

- IAM essentials for labs: users, roles, policies, permissions boundaries, least-privilege model.
- Amazon S3 fundamentals: buckets, objects, prefixes, event notifications, versioning, lifecycle rules.
- AWS Lambda fundamentals: runtime, handler, timeout, memory, IAM role, layers, cold/warm start.
- Amazon CloudWatch: log groups, log streams, metrics, basic alarms.
- Event-driven design basics relevant to pipeline workloads.

Hands-on labs

- Sign in to the AWS sandbox; verify issued IAM user, region, and permissions boundary.
- Inspect the S3 layout under your learner prefix (raw, validated, curated, output, quarantine zones).
- Deploy the starter Lambda; trigger it from a manual S3 upload.
- Read the execution log in CloudWatch; identify cold-start signature and memory usage.

Module 3 — Agentic AI Foundations for Engineering Teams

Field	Detail
Duration	4 hours ~30–45 min concepts, ~45–60 min guided design, ~2–2.25 hrs labs, ~20–30 min review
Track	Foundation
Module goal	Deepen the AI-assisted engineering model so participants can use AI for architecture, scaffolding, testing, and review — not just code generation.

Concepts covered

- Prompt engineering for technical work: role, constraints, examples, non-goals, output format.
- Context engineering: business rules, folder structure, coding standards, payload examples.
- Generating multiple architecture options with pros and cons using Azure OpenAI.
- Common failure modes in AI-generated code and designs (hallucinated APIs, missing error paths, over-abstraction).
- Review checklist for correctness, security, observability, scalability, and cost.

Hands-on labs

- Draft a prompt for an airline event processing flow that includes payload example, coding standards, and non-goals.
- Ask Azure OpenAI for two or three architecture variants; compare on a trainer-provided scorecard.
- Use GitHub Copilot Chat to critique a deliberately weak AI-generated function and suggest improvements.
- Refine the winning prompt based on review and re-run.

Module 4 — Batch Pipeline Design on AWS

Field	Detail
Duration	4 hours ~30–45 min concepts, ~45–60 min guided design, ~2–2.25 hrs labs, ~20–30 min review
Track	Batch
Module goal	Design a batch ingestion and transformation pipeline for airline ticket-issue files using auditable zone patterns, before writing any processor code.

Concepts covered

- Raw → validated → curated zone model on S3 and why each zone exists (audit, rerun, lineage).
- File-driven pipeline triggers: S3 event notifications, event filtering, sync vs async invocation.
- Validation strategy for airline batch files: schema, required-field checks, referential integrity.
- Choosing Lambda vs Glue for transformation steps.
- Auditability, reruns, and operational traceability.

Hands-on labs

- Ingest a synthetic airline ticket-issue batch file into the raw zone.
- Design the validation rule set on paper (airline code, fare class, OD pair, fare break, PNR format).
- Classify valid and invalid records and decide the routing for each.
- Draft a BRD for the batch processor using Azure OpenAI; refine through HITL review.

Module 5 — Agentic Batch Processing Implementation

Field	Detail
Duration	4 hours ~30–45 min concepts, ~45–60 min guided design, ~2–2.25 hrs labs, ~20–30 min review
Track	Batch

Field	Detail
Module goal	Implement the batch processor designed in Module 4, with modular coding standards, structured logging, error classification, and tests.

Concepts covered

- Coding standards for Lambda-based processors: folder structure (handler, service, validator, config, tests), no hardcoding, config-driven environments.
- Structured JSON logging and correlation IDs.
- Error classification: recoverable vs non-recoverable, quarantine routing, DLQ considerations.
- Retry and reprocessing considerations for batch flows (idempotency, poison-message handling).
- Writing unit tests for happy path, schema failure, and missing-metadata cases.

Hands-on labs

- Scaffold the processor module tree in VS Code using Copilot; enforce the standards-driven folder structure.
- Implement validation rules from Module 4; route valid records to validated/ and invalid to quarantine/.
- Add structured logging with correlation IDs.
- Write unit + functional tests; run against the synthetic dataset end-to-end.

Module 6 — Data Transformation, Cataloguing & Analytics

Field	Detail
Duration	4 hours ~30–45 min concepts, ~45–60 min guided design, ~2–2.25 hrs labs, ~20–30 min review
Track	Batch → Analytics
Module goal	Take processed airline data into queryable structures for reporting, exception handling, and settlement reconciliation — the kind of views Accelya’s platform exposes downstream.

Concepts covered

- Normalisation and enrichment of airline records (joining ticket, payment, and agent dimensions).
- Glue Data Catalog concepts: databases, tables, partitions, crawlers vs explicit registration.
- Athena query model, partition pruning, columnar formats (Parquet), cost-aware querying.
- Exception reporting for settlement and remittance: matched vs unmatched, delayed remittance,

adjustment deltas.

- Data layout choices that reduce scan cost (partitioning by settlement_date, airline_code).

Hands-on labs

- Transform validated ticket-issue records from Module 5 into a curated Parquet dataset partitioned by settlement_date and airline_code.
- Register the curated dataset in the Glue Data Catalog; run an Athena SELECT and inspect scan cost.
- Use Copilot Chat with an airline-aware prompt to draft an “unmatched remittance” Athena query; review and correct.
- Build a settlement reconciliation view joining remittance against ticket-issue records.

Module 7 — Streaming Architecture on AWS

Field	Detail
Duration	4 hours ~30–45 min concepts, ~45–60 min guided design, ~2–2.25 hrs labs, ~20–30 min review
Track	Streaming
Module goal	Introduce near real-time processing for airline events — booking, cancellation, refund — using Kinesis Data Streams with a Lambda consumer.

Concepts covered

- Streaming patterns and when to use them over batch for airline SLAs.
- Amazon Kinesis Data Streams fundamentals: shards, partition keys, ordering guarantees, retention.
- Event contracts: payload schema, versioning, backward compatibility.
- Partitioning strategy for airline events (by airline_code, by PNR).
- Integrating streaming consumers with downstream serverless components.

Hands-on labs

- Stand up a Kinesis Data Stream with a trainer-recommended shard count.
- Publish a stream of synthetic booking and cancellation events using a generator script.
- Consume events with a Lambda function; write consumed events to S3 for inspection.
- Observe stream-driven processing behaviour: batch size, iterator age, partition-keyhot-spotting.

Module 8 — Agentic Real-Time Processing Patterns

Field	Detail
Duration	4 hours ~30–45 min concepts, ~45–60 min guided design, ~2–2.25 hrs labs, ~20–30 min review
Track	Streaming
Field	Detail
Module goal	Use AI to explore and implement real-time processing patterns that are modular, reviewable, and cost-aware; layer in retries, idempotency, and DLQ behaviour.

Concepts covered

- Direct Consumer vs Orchestrator pattern for streaming: generated with AI, selected via HITL.
- Service modules for validation, enrichment, rule evaluation, routing, and response.
- Idempotent processing: why “exactly-once” is a myth at scale and how to design consumers that tolerate replay.
- Retry paths, dead-letter queues, and replay considerations with Lambda event source mapping.
- Balancing simplicity, scale, and cost in real-time systems.

Hands-on labs

- Generate event-processor variants with Azure OpenAI; select a pattern using HITL review criteria.
- Implement a modular event processing flow (validation → rule evaluation → output).
- Configure a DLQ; inject deliberately malformed events and observe retry / replay behaviour.
- Add CloudWatch alarms on consumer iterator-age and DLQ message count.

Module 9 — Real-Time ML/DL Inference on AWS

Field	Detail
Duration	4 hours ~30–45 min concepts, ~45–60 min guided design, ~2–2.25 hrs labs, ~20–30 min review
Track	Inference
Module goal	Deploy a lightweight model on SageMaker Serverless Inference and invoke it from the Module 8 streaming consumer to produce real-time refund recommendations or anomaly flags on cancellation events.

Concepts covered

- When to use Lambda-hosted logic vs SageMaker inference: latency, cold starts, model size, concurrency.
- Serverless inference patterns for low-volume classroom workloads.
- Passing stream events to an inference endpoint: feature preparation, timeout handling.
- Interpreting model output in business-friendly form (auto-approve / manual review / reject).
- Controlling cost and concurrency in Serverless Inference.
- Observability for inference: logging prediction, features, and model version on every call.

Hands-on labs

- Deploy a pre-trained lightweight refund-recommendation model (provided as a lab artefact) to a SageMaker Serverless Inference endpoint.
- Extend the Module 8 Lambda consumer to prepare features from the cancellation event and invoke the endpoint.
- Capture the prediction; persist it to an inference-output S3 bucket; emit a CloudWatch metric per recommendation class.
- Add a fallback path: if the endpoint errors or times out, route the event to a manual-review queue.

Module 10 — Observability, Security & Cost Optimisation

Field	Detail
Duration	4 hours ~30–45 min concepts, ~45–60 min guided design, ~2–2.25 hrs labs, ~20–30 min review
Track	Production-readiness
Module goal	Make the Modules 5–9 solutions production-aware through logging, monitoring, tightened IAM, and cost controls.

Concepts covered

- CloudWatch dashboards, log insights queries, and useful metric/alarm patterns.
- IAM least-privilege patterns for training workloads: permissions boundaries, condition keys.
- Operational diagnostics for failed batch and streaming runs.
- Service cost drivers and guardrails (Kinesis shards, Athena scan, CloudWatch ingest, SageMaker concurrency).
- Daily cleanup and safe-reset practices for ephemeral environments.

Hands-on labs

- Build a CloudWatch dashboard covering the batch processor, streaming consumer, and inference endpoint.
- Add targeted alarms (quarantine rate, iterator age, DLQ depth, inference error rate).
- Review and tighten IAM roles on the per-learner resources using least-privilege conditions.
- Run a cost-optimisation checklist against the built solution and capture the findings.

Module 11 — End-to-End Airline Case Study

Field	Detail
Duration	4 hours ~30–45 min concepts, ~45–60 min guided design, ~2–2.25 hrs labs, ~20–30 min review
Track	Capstone
Module goal	Connect the Accelya-aligned airline flows into one integrated solution spanning batch, streaming, and inference — the exact chain from booking to settlement.

Concepts covered

- Ticket booking data flow and event mapping (search → offer → PNR → ticket issue → payment).
- Cancellation and refund flow with rule evaluation and inference.
- Settlement and remittance adjustment outputs.
- Data lineage across the end-to-end pipeline.
- Operational reporting and exception visibility.

Hands-on labs

- Wire booking events (Modules 7–8) into ticket-issue batch processing (Modules 5–6).
- Route cancellation events through rule evaluation (Module 8) and inference (Module 9).
- Produce refund → settlement adjustment outputs into the curated analytics layer (Module 6).
- Generate a reconciliation + exception Athena report across the whole flow.

Module 12 — Capstone Review & Expert Evaluation

Field	Detail
Duration	4 hours ~30–45 min concepts, ~45–60 min guided design, ~2–2.25 hrs labs, ~20–30 min review

Track	Capstone
Module goal	Consolidate learning through participant walkthroughs, design critique, and an individual readiness evaluation.

Concepts covered

- Architecture quality checklist applied to the Module 11 solution.
- Prompt quality and context quality review — what made some AI-generated outputs better than others.
- Code review, test review, and improvement backlog.
- What would change for a production-grade implementation (multi-account, CI/CD, SLOs).
- Rollout considerations and a realistic path from workshop to production.

Hands-on labs

- Team / pair presents their Module 11 solution (10-minute walkthrough each).
- Peer review round: each team reviews another team's build against the production-readiness checklist.
- Trainer-led critique with targeted feedback on architecture, code, tests, observability, and cost.
- Individual readiness check: short written exercise applying the course review checklist to an unseen reference architecture.

5. Pre-Reads & Participant Readiness

The pre-read is designed for the mixed audience described in your email. It is split into mandatory reading (the foundation band must cover this before Module 1) and recommended reading (brings the working band closer to the DevOps band on specific topics).

5.1 Mandatory pre-reads (every participant)

- **AWS basics.** The official AWS getting-started pages for IAM, Amazon S3, AWS Lambda, and Amazon CloudWatch. Focus on: what each service is, how it charges, and how event notifications flow from S3 to Lambda.
- **JSON + Python refresher.** Reading a JSON event, accessing nested keys, writing a basic Lambda handler(event, context) signature, and emitting structured log output. Python 3.11+syntax.
- **Batch vs streaming vs inference.** A short conceptual read on when each pattern is appropriate — this is a recurring theme from Module 1 onwards.
- **Airline glossary.** The glossary and demo flows shared by Accelya (airline, airline code, market, Amadeus, ticket, fare class, fare break, OD, offer, cancellation/refund rules, no show, remittance, settlement). Terms are used across every module.
- **GitHub Copilot basics.** A 15-minute walkthrough on inline suggestions, Copilot Chat, and how to write Copilot-friendly prompts. Any participant already using Copilot daily can skip this.

5.4 Participant environment readiness

- Laptop with a modern browser, stable internet, permission to access AWS and Azure training accounts.
- VS Code installed (primary) with the GitHub Copilot extension signed in; Windsurf may be used as the alternate IDE.
- Python 3.11 or newer and Git installed locally for any offline exercises.
- AWS CLI v2 installed and configured only if the participant receives sandbox credentials.
- Access to Azure OpenAI via the endpoint and key provisioned by the trainer — no personal OpenAI keys are used.

5.5 Pre-course check-in (optional, recommended)

We recommend to confirm sandbox access for every participant, distribute the starter repository, walk through the glossary quickly, and answer setup questions. This measurably reduces Module 1 friction, especially for the foundation band.

6. Case Studies & Business Scenarios

The case studies below are derived directly from the two demo flows Accelya shared. They are used as the business backbone of the labs — the prompts, payloads, validation rules, and reports are all written against these flows. A third, integrated scenario is the capstone (Modules 11–12), so the course closes with a flow that matches Accelya’s settlement-oriented business context.

6.1 Case Study 1 — Ticket Booking Data Flow

Business flow

1. Passenger searches a route (e.g. BOM → DEL); the search request goes to the GDS (Amadeus).
2. GDS returns Offers — price, fare class, and fare rules.
3. Passenger selects an offer (e.g. Economy Saver).
4. PNR is created; seats are reserved.
5. Ticket is issued carrying fare class, fare break, and OD.
6. Payment is processed via the payment gateway.
7. Agent remits to airline; Accelya-class systems handle settlement and reconciliation.

How this flow maps into the course

Module(s)	What the learner builds	Airline flow stage covered
Modules 4–5	Batch processor for ticket-issue files.	Ticket Issuance.
Module 6	Curated dataset + Athena reconciliation views.	Remittance & Settlement views.
Modules 7–8	Streaming consumer for booking events.	Search, Offer, Selection, PNR Creation.
Modules 11–12	Integrated flow end-to-end.	Entire booking → settlement chain.

Synthetic fields used

- airline_code, market, OD (origin, destination), fare_class, fare_break indicator, offer_id, pnr, ticket_number, payment_status, remittance_reference, settlement_batch_id, booking_timestamp, issued_at, agent_id.

6.2 Case Study 2 — Cancellation & Refund Flow

Business flow

1. Passenger or agent initiates a cancellation request.
2. System fetches ticket details — fare class, OD, fare rules.
3. Rule evaluation: before or after departure, no-show status, applicable cancellation rules.
4. Refund calculation based on fare class, penalties, and refund rules (e.g. Saver fare → penalty; Flex fare → full refund).
5. Refund processed via payment system.
6. Settlement adjustment: agent balances, airline revenue, and settlement reports are updated.

How this flow maps into the course

Module(s)	What the learner builds	Airline flow stage covered
Modules 7–8	Kinesis stream + Lambda consumer for cancellation events; rule evaluation in code.	Cancellation Request, Fetch Ticket, Rule Evaluation.
Module 9	SageMaker inference invocation from the streaming consumer for refund recommendation / anomaly flagging.	Refund Calculation, Anomaly Detection.
Modules 11–12	Refund → settlement adjustment output integrated with the booking flow.	Refund Processing, Settlement Adjustment.

Synthetic fields used

- ticket_id, fare_class, OD, journey_date, departure_status, no_show_flag, cancellation_timestamp, penalty_table_ref, refund_rule_id, refund_amount, payment_reference, settlement_delta, adjustment_reason_code.

6.3 Integrated Capstone Scenario — Settlement & Exception Intelligence

Modules 11–12 combine outputs from booking, ticketing, cancellation, refund, remittance, and settlement into a reconciliation layer that flags mismatches, delayed remittance, and suspicious adjustment patterns — the exact kind of exception surface Accelya’s platform exists to produce. This extension uses only the synthetic fields already in play across Modules 4–10, plus a small anomaly marker set added for the capstone.

Additional synthetic fields introduced in the capstone

- remittance_due_date, remittance_received_date, remittance_delay_days, adjustment_category, anomaly_marker, reconciliation_status.

7. Lab Setup Guide

The lab is intentionally trainer-managed and serverless-first. This model minimises setup friction for the foundation band, keeps cost predictable, and lets the senior band focus on architecture rather than account plumbing.

7.1 Provisioning model

Aspect	Decision
Account model	Trainer-managed AWS sandbox account with one dedicated IAM user per learner under a shared account, each with a per-learner S3 prefix and permissions boundary.
Region	Single fixed region for the entire batch (recommended: ap-south-1 Mumbai) to avoid cost fragmentation and cross-region surprises.
IDE	VS Code (primary) with GitHub Copilot; Windsurf as alternate. Both used against the same starter repository.
Model access	Azure OpenAI endpoint and key provisioned by the trainer; shared via a course-scoped secret.
Source control	GitHub repository (private) with branch-per-learner; optional GitHub Codespaces for zero-setup participants.

7.2 Operational guardrails

- No public S3 buckets anywhere in the course — all access is identity-based.
- No persistent SageMaker notebook instances; inference is Serverless.
- Kinesis streams are provisioned only for Modules 7–12 and deleted at end of Module 12.
- AWS Budgets alarm configured at the batch level; cleanup script run at the end of each module.
- Consistent naming standard: accelya-m<module>-<learner>-<resource> so stragglers can be found and cleaned up.
- Azure OpenAI call volume is capped at the course key level; trainer monitors daily.

7.3 Quick-start checklist for learners (Module 1, first 30 minutes)

#	Learner action	Expected result
1	Sign in to the AWS console using issued credentials and fixed region.	Console dashboard visible; correct region shown.

#	Learner action	Expected result
2	Open VS Code / Windsurf; clone the starter repository; verify GitHub Copilot sign-in.	Repo visible; Copilot suggestions active.
3	Verify S3 buckets and your learner prefix are accessible.	Raw, validated, curated, output folders listed.
4	Invoke the starter Lambda and view logs in CloudWatch.	Baseline execution confirmed.
5	Load the Module 1 synthetic dataset snippet into your raw/ prefix.	Module 1 lab starting state reached.
6	Confirm Azure OpenAI call works from the course script.	Endpoint, key, and model name validated.

8. Synthetic Data | Ownership, Scope & Open Question

Open question for Accelya — please confirm

Accelya has clearly indicated that no real production or customer data will be shared. That is expected and fully supported by this design. What we need to confirm before Day 0 is who generates the synthetic data the labs run on. Two options are on the table:

1. Option 1 — Jai Singh generates synthetic data. In this case, Accelya shares only non-sensitive reference artefacts — field lists, schemas, code tables, sample payload structures, and validation rules. No actual records are required. The trainer generates internally consistent synthetic records against those schemas.
2. Option 2 — Accelya provides fully anonymised synthetic data. In this case Accelya ships the synthetic records and Jai Singh only adapts them into the lab folder structure.

Either option works.

8.1 What Accelya is expected to share (Option 1, default)

- Field lists for booking, ticket issue, cancellation, refund, remittance, and settlement records.
- Code tables and enumerations: airline codes, market codes, fare class codes, adjustment reason codes, exception category codes.
- Sample payload structures (JSON or CSV schemas) — no actual records required.
- Validation rules that real Accelya pipelines apply (format checks, cross-field rules, referential constraints).
- A list of business terms that must appear verbatim in schemas and log fields so participants see Accelya-authentic language.

8.2 Data-volume pricing (additional charge trigger)

For larger datasets

The default synthetic dataset is sized to support the full 12-module program on a cohort of up to 15 learners - typically a few records per flow (booking, ticketing, cancellation, refund, remittance, settlement), scaled so Athena queries and Kinesis playback remain realistic but cheap.

If Accelya requires a materially larger dataset — for example thousands of records per flow, an additional synthetic-data-generation charge applies to cover schema alignment, generator extension, internal-consistency validation, and regeneration support.

8.3 Deliverables bundled with the synthetic data pack

- Synthetic CSV and JSON datasets for both use cases plus the capstone.
- Data dictionary describing every field, type, allowed values, and business meaning.
- Scenario catalogue covering normal path, failure paths, and edge cases (e.g. no-show, fare-class-change, delayed remittance).
- Regeneration scripts for all datasets.

9. Deliverables, Assumptions & Next Steps

9.1 Deliverables from Jai Singh

- This proposal document (revised ToC, pre-reads, case studies, lab setup, pricing, synthetic data strategy).
- Starter GitHub repository with modular Lambda code, SAM templates, prompts, and test skeletons.
- Synthetic data pack (datasets, dictionary, regeneration scripts).
- Azure OpenAI prompt library for BRD drafting, architecture generation, and test generation.
- Trainer-side AWS sandbox: IAM, S3 zones, CloudWatch log groups, Glue Data Catalog stub, Athena workgroup, CloudWatch dashboards, cleanup scripts.
- Pre-trained refund-recommendation model artefact for Module 9.
- Quick-start handout, per-module lab instructions, and end-of-module cleanup script.
- Attendance and engagement report per module; end-of-program readiness scorecard per participant.

9.2 Assumptions

- Cohort size: up to 15 learners per batch. Larger cohorts re-priced per Section 8.4.
- Delivery cadence: 12 half-day modules of 4 hours each. Exact schedule agreed with Accelya before Day 0.
- Single fixed AWS region for the whole batch.
- Microsoft + AWS toolchain only — VS Code / Windsurf, GitHub Copilot, Azure OpenAI, and AWS services as listed. No Google and no non-approved OpenAI endpoints.
- Accelya confirms the synthetic data option (Section 9) before Day 0.
- Delivery is instructor-led; hybrid (in-person + remote) is supported with adequate A/V.
- Start date targeted for 11 May 2026, subject to confirmation of pre-course readiness.

9.3 Open items requiring a response from Accelya

1. Synthetic data ownership — Option 1 (trainer-generated) or Option 2 (client-provided). Default: Option 1.
2. Lab model — Option A (shared managed lab, recommended) or Option B (dedicated sandbox per participant).
3. Cohort size for the first batch and whether multiple batches are planned.
4. Preferred cadence — 5, 3, or 2 modules per week (see Section 3.2) and morning or afternoon slot.
5. Whether a pre-course 30-minute check-in is acceptable 3–5 days before Module 1.
6. Target dataset volume — default “few thousand records per flow” or larger (triggers the charge).

9.4 Next steps

1. Accelya confirms the six open items above.
 2. Jai Singh issues the commercial order with the confirmed optionset.
 3. Day-0 provisioning begins 5–7 working days before the first module.
 4. Optional pre-course check-in 3–5 days before Module 1.
 5. Program delivery — 12 half-day modules × 4 hours, starting 11 May 2026 (subject to confirmation).
-